

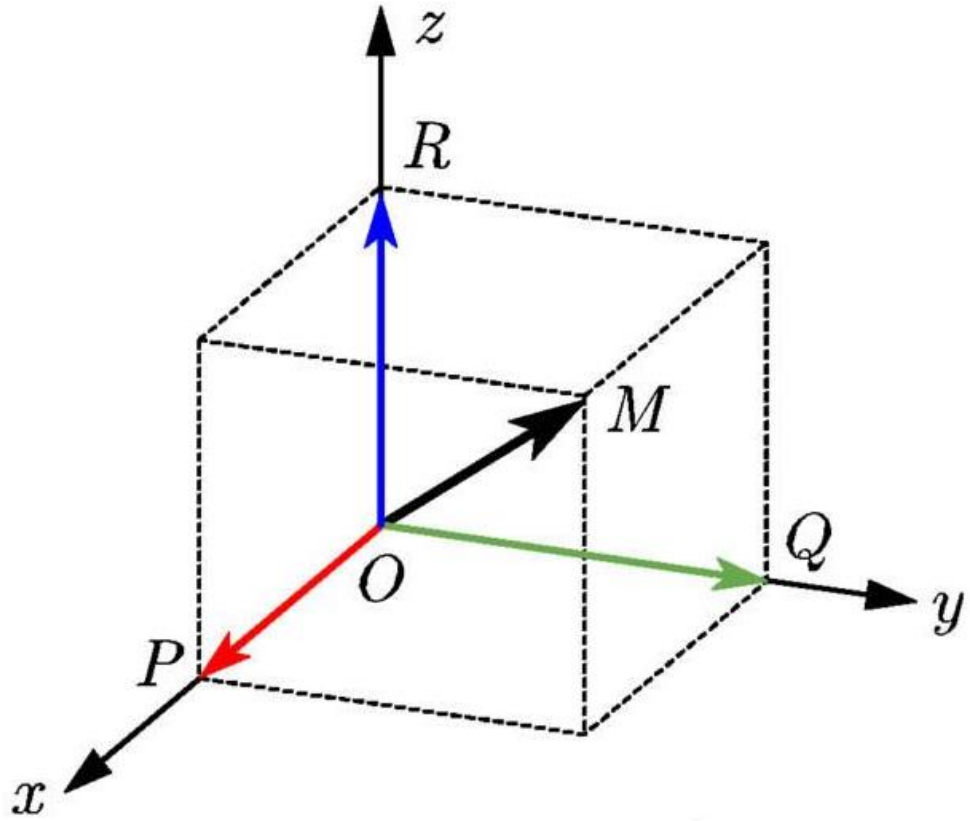
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

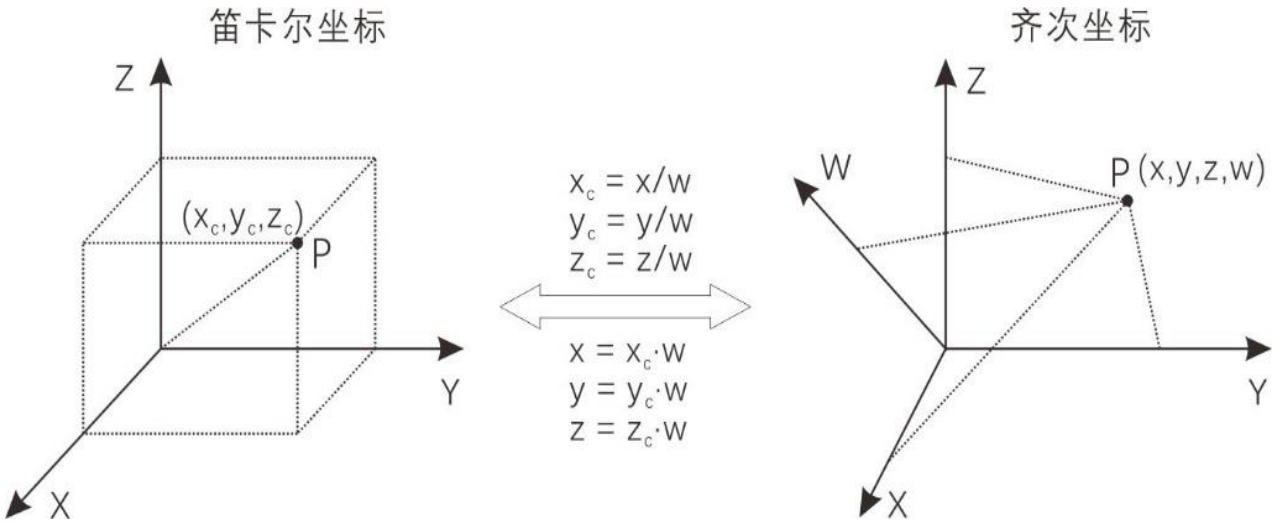
第2章 基础几何库搭建

- **内容提要**
  - 点、向量、矩阵
  - 直线、射线、线段
  - 多段线
  - 平面
  - 几何算法



1. 点、向量、矩阵

- **点**
  - 用**笛卡尔坐标**表示一个**三维点**：(x_c, y_c, z_c)
  - 用**齐次坐标**表示一个**四维点**：(x, y, z, w)
  - 笛卡尔坐标和齐次坐标的转化关系如图所示：



例

笛卡尔点(2, 3, 4)的齐次坐标可以是(2, 3, 4, 1)，也可以是(4, 6, 8, 2)。

注意：

- 对齐次坐标(x，y，z，w)，当w不为0时，它总是对应一个唯一的笛卡尔坐标；但当w为0时，上述转换关系式将出现“零除”错误。为此，规定当齐次坐标中的最后一个维度w为0时，该点表示**无穷远点**。
- 在用齐次坐标表示一个点时，一般情况下，如果该点不是一个无穷远点，习惯将w设为1，即(x, y, z, 1)，此时齐次坐标和笛卡尔坐标在对应维度数值上保持一致，便于理解。

为什么要用齐次坐标表示三维点?

1. 点、向量、矩阵

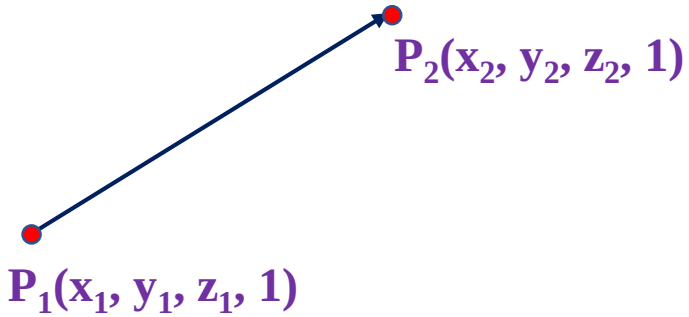
• 向量

- **在计算几何中，向量是反映两个点之间的一种关系量。**
- **点有位置**的概念，而**向量**没有位置，**只有方向和长度**（模）的概念，向量在空间中经任意平移后，几何意义不变，因此可以认为向量来自无穷远处。
- 向量也可以用**齐次坐标**表示，在定义一个三维向量的齐次坐标(dx, dy, dz, dw)时，规定**dw = 0**，即(dx, dy, dz, 0)。
- 向量可以看成是空间中一个点$P_1(x_1, y_1, z_1, 1)$到另一个点$P_2(x_2, y_2, z_2, 1)$的**增量部分**，如果用V表示这个增量，那么V的坐标为$(x_2-x_1, y_2-y_1, z_2-z_1, 0)$。对向量和点，可以定义加号、减号操作符（‘+’、‘–’），使得：

$$V = P_2 - P_1, \quad (2\text{-}1)$$

$$P_2 = P_1 + V, \quad (2\text{-}2)$$

$$P_1 = P_2 - V. \quad (2\text{-}3)$$

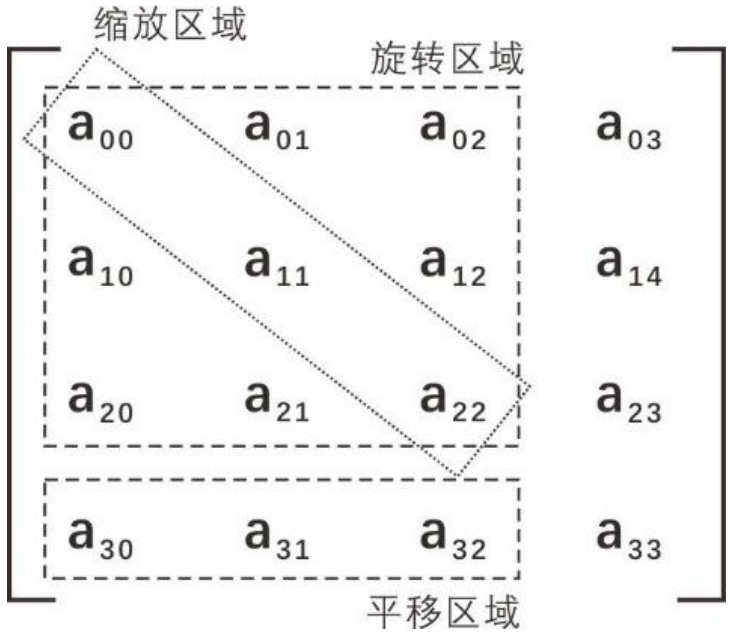


- 公式(2-1)表示一个向量可以通过两个点相减操作得到，向量的指向是从P_1指向P_2；
- 公式(2-2)和公式(2-3)表示一个点可由另一个点加上（或减去）一个向量得到，事实上就是一个点可以通过另一个点**平移**得到。

1. 点、向量、矩阵

• 矩阵

- **在计算几何中，矩阵是反应点和点、向量和向量以及点和向量之间的复杂关系量。**
- 点和向量可以看作是一个**1行4列**的矩阵，对点和向量的几何变换可由该点或向量**右乘**一个矩阵得到。
- 在齐次坐标下，一个矩阵将包含**4行4列**共16个元素。这16个元素又可分为3个子区域，其中：



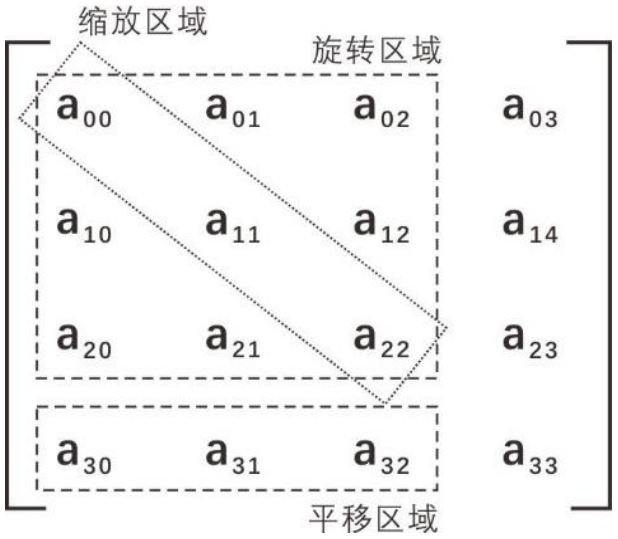
说明：

- 左上角的9个元素（a_{00}~a_{22}）用于**旋转变换**，
- 下方3个元素（a_{30}、a_{31}、a_{32}）用于**平移变换**，
- 对角线3个元素（a_{00}、a_{11}、a_{22}）用于**缩放变换**。

1. 点、向量、矩阵

• 矩阵：平移

- 设平移矩阵为$\mathbf{M_t}$，构造$\mathbf{M_t}$只需对矩阵下方的a_{30}、a_{31}、a_{32}这3个元素赋相应维度的平移量（t_x、t_y、t_z）。
- 对任意点P(x, y, z, 1)，其平移变换过程如公式(2-4)所示。变换后，得到的新点**P'**坐标为$(x+t_x, y+t_y, z+t_z, 1)$。



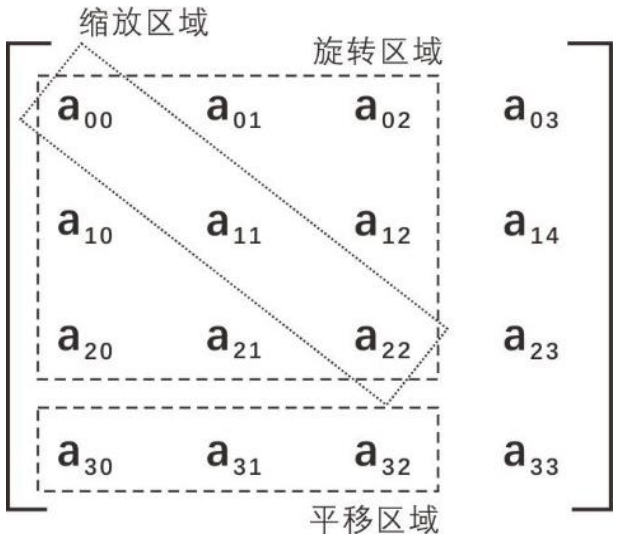
$$P' = P \cdot M_t$$

$$= (x, y, z, 1) \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix} \qquad (2\text{-}4)$$

$$= (x + t_x, y + t_y, z + t_z, 1)$$

1. 点、向量、矩阵

- **矩阵：缩放**
- 设缩放矩阵为$\mathbf{M}_s$，构造$\mathbf{M}_s$只需对矩阵中的a_{00}、a_{11}、a_{22}对角线3个元素赋相应维度的缩放量（s_x、s_y、s_z）。
- 对任意点P(x, y, z, 1)，其平移变换过程如公式(2-5)所示。变换后，得到的新点**P'**坐标为$(x\cdot s_x, y\cdot s_y, z\cdot s_z, 1)$。



$$P' = P \cdot M_s$$

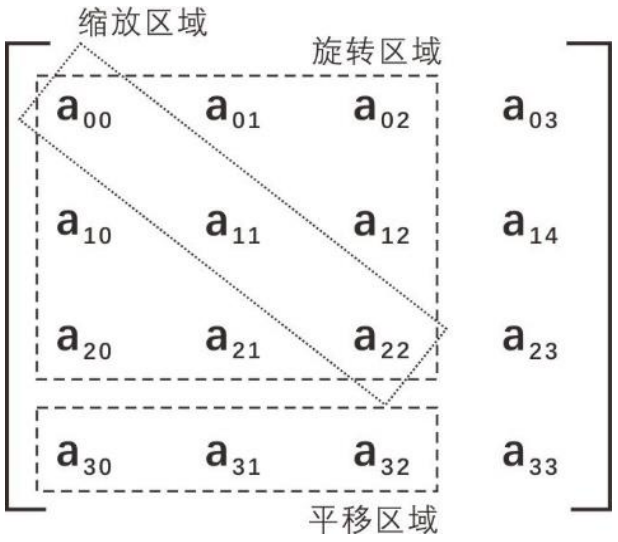
$$= (x, y, z, 1) \cdot \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad (2\text{-}5)$$

$$= (x \cdot s_x, y \cdot s_y, z \cdot s_z, 1)$$

1. 点、向量、矩阵

• 矩阵：旋转

- 因为点绕空间中任意轴的旋转都可以看成是该点依次绕X轴、Y轴、Z轴三个轴旋转的合成，因此这里分别介绍绕X轴、Y轴、Z轴旋转的矩阵。设绕X轴、Y轴、Z轴旋转的矩阵分别为$\mathbf{M}_{rx}$、$\mathbf{M}_{ry}$、$\mathbf{M}_{rz}$，旋转角度为θ，则：



$$M_{rx}(\theta)=\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & \cos\theta & \sin\theta & 0\\ 0 & -\sin\theta & \cos\theta & 0\\ 0 & 0 & 0 & 1\end{bmatrix}, \tag{2-6}$$

$$M_{ry}(\theta)=\begin{bmatrix}\cos\theta & 0 & -\sin\theta & 0\\ 0 & 1 & 0 & 0\\ \sin\theta & 0 & \cos\theta & 0\\ 0 & 0 & 0 & 1\end{bmatrix}, \tag{2-7}$$

$$M_{rz}(\theta)=\begin{bmatrix}\cos\theta & \sin\theta & 0 & 0\\ -\sin\theta & \cos\theta & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix}. \tag{2-8}$$

1. 点、向量、矩阵

- **矩阵：旋转**
  - 以$\mathbf{M}_{rz}$矩阵为例，点P(x, y, z, 1)绕Z轴旋转θ角度的变换过程：

$$
\begin{aligned}
P' &= P \cdot M_{rz}(\theta) \\
&= (x, y, z, 1) \cdot \begin{bmatrix} \cos\theta & \sin\theta & 0 & 0 \\ -sin\theta & cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= (x\cos\theta - y\sin\theta, x\sin\theta + y\cos\theta, z, 1)
\end{aligned} \tag{2-9}
$$

- **方向规定：**
  - 如果θ为**正**，则按**逆时针**旋转；
  - 如果θ为**负**，则按**顺时针**旋转。

1. 点、向量、矩阵

• 矩阵：综合变换

- 在实际应用中，对一个点的几何变换往往是包含平移、缩放、旋转中一个或多个的**复合操作**，这时，就要按照操作的顺序**依次对点右乘相应矩阵**，而且**右乘的次序不能随意调换**，否则会影响几何变换结果。
- 如对式(2-10)，首先对点P绕Z轴旋转θ_1角度，然后再对得到的更新点绕Y轴旋转θ_2角度，随后分别对得到的更新点平移和缩放，最后得到P'。

$$P' = P \cdot M_{rz}(\theta_1) \cdot M_{ry}(\theta_2) \cdot M_t \cdot M_s \qquad (2\text{-}10)$$

1. 点、向量、矩阵

- **几何基础模块实现：GeomBase**
- 使用Python对的点、向量和矩阵进行封装，创建3个类，分别为：**Point3D**、**Vector3D**、**Matrix3D**，并在这些类中定义相关函数。

GeomBase.py	几何基础库，定义点、向量、矩阵以及它们之间的相互操作	2-1

```
1   import math                                  # 导入 math 库
2   epsilon = 1e-7                               # 定义一个极小值，10-7
3   epsilonSquare = epsilon * epsilon            # 极小值平方
4   class Point3D:                               # 定义点类
5       def __init__(self, x=0.0, y=0.0, z=0.0, w = 1.0):
6           self.x, self.y, self.z, self.w = x, y, z, w
7   class Vector3D:                              # 定义向量类
8       def __init__(self, dx = 0.0, dy = 0.0, dz = 0.0, dw = 0.0):
9           self.dx, self.dy, self.dz, self.dw = dx, dy, dz, dw
10  class Matrix3D:                              # 定义矩阵类
11      def __init__(self):
12          self.a = [[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1]]
```

略，参见PyCharm中代码

1. 点、向量、矩阵

- **几何基础模块解析：Point3D**

- **clone**函数，即在内存中复制一个和当前点一模一样的点，当我们修改原来的点坐标，新点的坐标不会随之改变。
- **pointTo**函数，它的字面意思为“指向”，传入参数为另一个点，这个函数的作用是计算并返回从当前点到另一个点的向量，测试代码如下：

	Point3D类pointTo函数测试	2-2
1	p1 = Point3D(1, 1, 1)	# 创建第1个点
2	p2 = Point3D(2, 3, 4)	# 创建第2个点
3	v = p1.pointTo(p2)	# 通过pointTo函数构造向量v
4	print(v)	# 显示Vector3D: 1, 2, 3

- **translate**函数和**translated**函数的作用是对当前点作用一个向量，使之沿该向量平移。但是translate函数作用于当前点本身，作用后，当前点坐标发生变化；而translated函数将返回一个新点，当前点坐标不变。

	Point3D类translate和translated函数测试，沿用以上p1、p2坐标	2-3
1	v = Vector3D(1, 1, 1)	
2	p1.translate(v)	
3	p3 = p2.translated(v)	
4	p4 = p2 + v	
5	print(p1, p2, p3, p4) # 结果p1坐标(2, 2, 2)；p2坐标(2, 3, 4)；p3坐标(3, 4, 5)；p4坐标(3, 4, 5)	

1. 点、向量、矩阵

- **几何基础模块解析：Point3D**

- **multiplied**函数的作用是对当前点右乘一个矩阵，返回新的点。需要强调的是，我们点和矩阵的相乘关系定义在Point3D类里，点在左边，矩阵在右边，因此点是右乘矩阵。

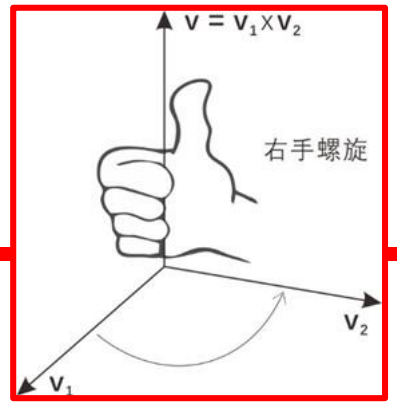
	Point3D类 ‘*’ 符号函数测试	2-4
1	p1 = Point3D(1, 1, 1)	
2	m = Matrix3D.createTranslateMatrix(1, 2, 3)	# 创建平移矩阵
3	p2 = p1 * m	# 点又乘矩阵得到另一个点
4	print(p2)	# 显示p2坐标为(2, 3, 4)

- **distance**和**distanceSquare**函数，都和距离计算相关，输入均为另一个点。其中distance计算两个点在三维空间中的真实距离，需要开根号；而distanceSquare计算两个点距离的平方，不需要进行根号运算。比较而言，后者运算速度更快。

- **isCoincide**和**isIdentical**两个函数，这两个函数用于判断两个点是否重合或完全一致。在isCoincide函数中，使用两个点的距离是否小于某一个给定的极小值（参见GeomBase.py开头的epsilon）来判断两个点是否重合（**截断误差**）；而在isIdentical函数中，使用两个点的各个维度上的坐标值是否分别相等来判断两个点是否一致。

1. 点、向量、矩阵

$$\mathbf{v_1} \times \mathbf{v_2} = \begin{bmatrix} i & j & k \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \end{bmatrix}$$



• 几何基础模块解析：Vector3D

- **reverse**和**reversed**函数表示对向量进行取反操作，即对向量的每个坐标值取反。
- **dotProduct**和**crossproduct**函数都需要输入另一个向量作为参数，分别表示对两个向量求**点乘**和**叉乘**，其中向量点乘返回一个实数，向量叉乘返回一个向量。
  - 向量点乘：如果两个向量的夹角小于90度，则点乘的结果为正；如果夹角大于90度，则点乘结果为负；如果夹角刚好为90度（即正交），则点乘结果为0。根据向量点乘这些特性，可以对任意两个向量的**夹角范围做出判断**。
  - 向量叉乘：对积向量，它的方向分别垂直于两输入向量，具体指向可根据右手法则确定。如果两个向量平行（方向相同或相反），那么这两个向量叉乘的积向量为零向量（即积向量长度为0）。根据向量叉乘的这一特性，可以对任意两个向量**是否平行做出判断**。
- **amplify**和**amplified**函数表示对向量进行放大操作，即对向量中的每个坐标值乘以一个给定实数。
- **normalize**和**normalized**函数表示对向量进行单位化操作，即对向量中的每个坐标值除以当前向量长度。
- **getAngle**函数用于计算当前向量和另一向量的夹角，夹角取值范围为0~π。对向量v1和v2，首先对两个向量进行单位化处理，然后计算两个向量的点乘值dotPro（取值范围-1~1），并使用反余弦函数计算夹角，参见式(2-11)：

$$\text{angle} = \arccos(\mathrm{v}_1 \cdot \mathrm{v}_2). \qquad (2\text{-}11)$$

1. 点、向量、矩阵

- **几何基础模块解析：Matrix3D**

- **makeIndentical**函数可对一个已经存在的矩阵进行单位化操作。函数**multiplied**返回当前矩阵和另一矩阵相乘的结果。
- **getDeterminant**函数用于计算矩阵的行列式，函数**getReverseMatrix**用于计算当前矩阵的逆矩阵（如果存在）
- 此外，在Matrix3D类中，定义了4个静态函数，分别为**createTranslateMatrix**、**createScaleMatrix**、**createRotateMatrix**和**createMirrorMatrix**，分别实现创建平移矩阵、创建缩放矩阵、创建旋转矩阵和创建镜像矩阵的功能。

```
1  mt = Matrix3D.createTranslateMatrix()      # 创建平移矩阵
2  ms = Matrix3D.createScaleMatrix()          # 创建缩放矩阵
```

2. 直线、射线、线段

• 直线

- 直线是空间中经过不重合的两个点且可向两端**无限延伸**、**曲率处处为零**的一种特殊曲线，它的长度无法度量。
- 平面几何中，直线可由**截距式**表示：

$$y = kx + b \tag{2-12}$$

- 立体几何中，直线可由**点向式**表示：

$$\frac{x-x_0}{m} = \frac{y-y_0}{n} = \frac{z-z_0}{l} \tag{2-13}$$

- 或**一般式**表示（表示两张平面交线）：

$$\begin{cases} A_1x + B_1y + C_1z + D_1 = 0 \\ A_2x + B_2y + C_2z + D_1 = 0 \end{cases} \tag{2-14}$$

2. 直线、射线、线段

• 直线

- 根据以上介绍，可以选择直线的点向式或一般式来表达一条直线，但显然，直线的**点向式**更加直观且容易让人理解，因此，这里我们使用直线的点向式来表示直线。
- 建立直线类，类名为**Line**，写在Line.py文件中，如下：

Line.py	直线类	2-5

```
1   from GeomBase import *                  # 导入GeomBase模块
2   class Line:
3       def __init__(self, P, V):
4           self.P = P.clone()              # 直线经过的点，类型为Point3D
5           self.V = V.clone().normalized() # 直线方向向量，类型为Vector3D，单位向量
6       def __str__(self):                  # 定义直线类被print时的显示行为
7           return "Line\nP %s\nV %s\n" %(str(self.P), str(self.V))
```

- **思考：**在Line类的__init__函数中，**并没有直接对P（或V）进行直接赋值**（如self.P = P），而是调用了点**clone**函数，将在内存中复制得到的新点赋给Line的P属性。**这是为什么？**

2. 直线、射线、线段

• 直线

- 以上调用clone函数赋值的原因是：**降低Line对象创建后，其属性在外部被修改的风险**。
- 这是初学者在使用类似Python、C#、Java等使用**引用方式传值**的编程语言时非常容易犯的错误，而且这样的bug有时很难被发现。可以运行以下简单的示例代码以便加深印象。

	Line属性外部修改测试，假设类的构造函数中对点赋值时不用clone函数	**2-6**
1	p = Point3D(1, 2, 3)	
2	v = Vector3D(4, 5, 6)	
3	ln = Line(p, v)　　# 根据p、v定义直线	
4	print(ln)　　# 显示P: 1, 2, 3　V: 4, 5, 6	
5	p. x = 0　　# 修改点p的x属性，观察ln的P属性是否被改变	
6	print(ln)　　# 显示P: 0, 2, 3　V: 4, 5, 6　这表明ln属性在外部被改变	

2. 直线、射线、线段

- **射线**
- 相比直线，射线更应该用**点向式**来表示，因此在实现射线类时，几乎可以照搬直线类的实现代码。
- 将射线类放在Ray.py文件中，类名为Ray。

Ray.py	射线类	2-7

```
1  from GeomBase import *
2  class Ray:
3      def __init__(self, P, V):
4          self.P = P.clone()                  # 射线经过的点，类型为Point3D
5          self.V = V.clone().normalized()             # 射线方向向量，类型为Vector3D，单位向量
6      def __str__(self):              # 定义射线类被print时的显示行为
7          return "Ray\nP %s\nV %s\n" %(str(self.P), str(self.V))
```

2. 直线、射线、线段

• 线段

- 线段长度固定，可由两端点定义。将线段类放在Segment.py文件中，类名为Segment。

Segment.py	线段类	2-8

```
1   from GeomBase import *
2   class Segment:
3       def __init__(self, A, B):
4           self.A, self.B = A.clone(), B.clone()          # 线段两端点A、B
5       def __str__(self):                                 # 线段被print时的行为
6           return "Segment\nA %s\nB %s\n" %(str(self.A), str(self.B))
7       def length(self):                                  # 计算线段长度
8           return self.A.distance(self.B)
9       def direction(self):                               # 计算线段方向
10          return self.A.pointTo(self.B)
11      def swap(self):                                    # 交换线段两端点
12          self.A, self.B = self.B, self.A
```

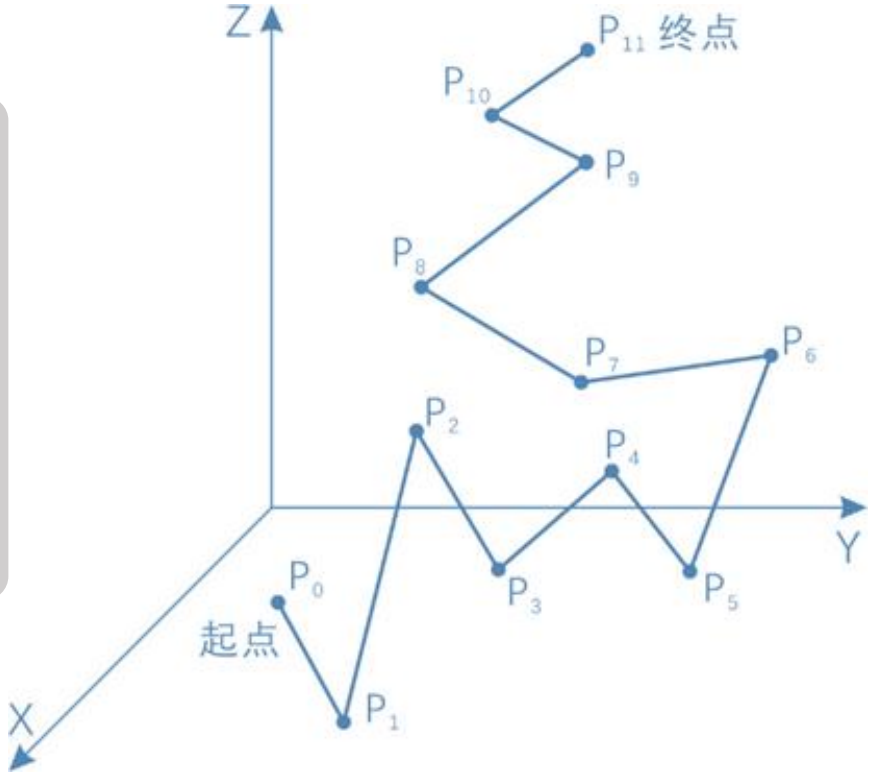
- 在线段类中，**length**函数用于计算线段长度，**direction**函数用于计算线段所在直线的方向，**swap**函数用于**交换**端点A和B的坐标（注意交换写法！）。

3. 多段线

- **定义：依次**连接空间中**多个顶点**得到的线型就是**多段线（Polyline）**。
- 多段线又叫多义线、折线或点列曲线（point-sequence curve, PS-curve）。

多段线特性

- 多段线中的顶点不是杂乱无序，而是**依次连接**的；
- **顶点个数至少为2**，当顶点数恰为2时多段线退化为线段。
- 多段线包含两个端点，如果这两个端点重合，那么称多段线是**闭合**的，反之则称多段线是**开放**的。
- 对一条闭合的多段线，如果它所有的顶点都落在同一平面（如XY平面）上，那么多段线退化为一个封闭的**平面多边形**。



3. 多段线

- 多段线类创建，类名为**Polyline**，对应的实现文件为Polyline.py。

Polyline.py　多段线类　2-9

```
1   from GeomBase import *
2   class Polyline:
3       def __init__(self):
4           self.points = []
5       def __str__(self):                  # 定义Polyline被print时的显示行为
6           if self.count() > 0:
7               return 'Polyline\nPoint number: %s\nStart %s\nEnd %s\n' \
8                   % (self. count(), str(self. startPoint()), str(self.endPoint()))
9           else: return 'Polyline\nPoint number: 0\n'
10      def clone(self):                    # 克隆一条多段线
11          poly = Polyline()
12          for pt in self.points:
13              poly.addPoint(pt.clone())
14          return poly
15      def count(self):                    # 获取多段线中点的数量
16          return len(self.points)
17      def addPoint(self, pt):             # 在多段线末尾添加一个点
18          self.points.append(pt)
19      def addTuple(self, tuple):          # 在多段线末尾添加一个点（元组形式）
20          self.points.append(Point3D(tuple[0], tuple[1], tuple[2]))
```

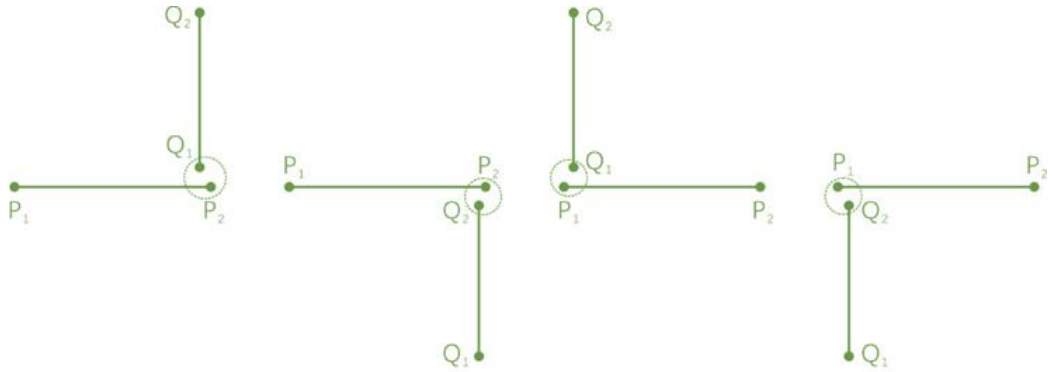
3. 多段线

- **addPoint**和**raddPoint**函数，作用都是向已有的多段线添加一个顶点，不同的是，addPoint向多段线的终点之后添加一个顶点，而raddPoint向多断线的起点之前插入一个顶点，raddPoint中的r表示反向（reverse）。
- **getArea**函数的作用是获取位于XY平面上的多边形面积，或者是对任意封闭多段线，获取它在XY平面上投影的面积。理论上，对XY平面上的任意多边形，只要它各顶点的位置确定，那么它的面积就已确定，计算公式为：

$$S = \frac{1}{2}\sum_{0}^{n-1}(P_i.x \cdot P_{i+1}.y - P_{i+1}.x \cdot P_i.y), \qquad (2\text{-}15)$$

- **注意**：**getArea**函数还可以用来判断多边形的方向。如果一个多边形面积为正，则其方向是**逆时针（CCW）**的；反之则其方向为**顺时针（CW）**。

- **appendSegment**函数的作用是将一条线段拼接到一条多段线上，前提是线段的两个端点和多段线的两个端点中必须有一对是重合的，即需要讨论端点拼接的四种情况。如果拼接成功，函数返回True，如果拼接失败，函数返回False



3. 多段线

- **全局函数writePolyline和readPolyline**的功能分别是将多段线写入自定义文本文件或从已保存的文本文件中读取多段线。文件格式如下，其中第1行为顶点个数，其余各行为顶点各x、y、z坐标。创建这两个函数的目的是方便今后调试时直接查看多段线中顶点数据。

1	1001
2	100, 200, 300
3	401, 321, 212
4	123, 456, 789
…	……

注意：

- **在这两个函数中，使用了Python的异常处理语句（try … except … finally），并对异常进行打印。**
- **在使用Python语言对计算机IO（如文件、网络、串口等）进行操作时，为了保证程序运行的健壮性，强烈建议在代码中添加异常处理语句。**

4. 平面

- 平面可以看成是空间中过某一给定点且垂直于某一给定向量的所有直线的集合，这个向量就是平面的法向量。因此：

$$A(x - x_0) + B(y - y_0) + C(z - z_0) = 0. \qquad (2\text{-}16)$$

- 式中(x_0, y_0, z_0)为一给定点，(A, B, C)为给定法向量，(x, y, z)为平面上任意一点。进一步地，上式可以转化为：

$$(A, B, C) \cdot (x - x_0, y - y_0, z - z_0) = 0. \qquad (2\text{-}17)$$

- 和直线、射线一样，可以根据一个**点**和一个**向量**来定义一张平面。将平面类放在Plane.py文件中，类名为Plane。

4. 平面

Plane.py	平面类	2-10

```
1   ……                                          # 导入相关库
2   class Plane:
3       def __init__(self, P, N):
4           self.P = P.clone()                  # P为平面经过的一点
5           self.N = N.clone().normalized()     # N为平面法向量，单位向量
6       def __str__(self):
7           return "Plane\n%s\n%s\n" % (str(self.P), str(self.N))
8       def toFormula(self):    # 获取标准直线方程 Ax+By+Cz+D=0 中参数A、B、C、D值
9           A, B, C = self.N.dx, self.N.dy, self.N.dz
10          D = -self.N.dx * self.P.x - self.N.dy * self.P.y - self.N.dz * self.P.z
11          return A, B, C, D
12      @staticmethod
13      def zPlane(z):                          # 静态函数，返回一个z平面
14          return Plane(Point3D(0, 0, z), Vector3D(0, 0, 1.0))
15      def intersect(self, other):             # 两平面求交，返回Line或None
16          dir = self.N.crossProduct(other.N)  # dir为直线方向向量
17          if dir.isZeroVector():
18              return None
19          else:
20              x, y, z = 0, 0, 0               # x, y, z为直线经过点坐标
21              A1, B1, C1, D1 = self.toFormula()   # 通过平面标准方程联立求解
22              A2, B2, C2, D2 = other.toFormula()
23              if B2*C1 - B1*C2 != 0:
24                  y = -(-C2*D1 + C1*D2)/(B2*C1 - B1*C2)
25                  z = -(B2*D1 - B1*D2)/(B2*C1 - B1*C2)
26              elif A2*C1 - A1*C2 != 0:
27                  x = -(-C2*D1 + 1*D2)/(A2*C1 - A1*C2)
28                  z = -(A2*D1 - A1*D2)/(A2*C1 - A1*C2)
29              elif A2*B1 - A1*B2 != 0:
30                  x = -(-B2*D1 + B1*D2)/(A2*B1 - A1*B2)
31                  y = -(A2*D1 - A1*D2)/(A2*B1 - A1*B2)
32              else: return None
33              return Line(Point3D(x, y, z), dir.normalized())
```

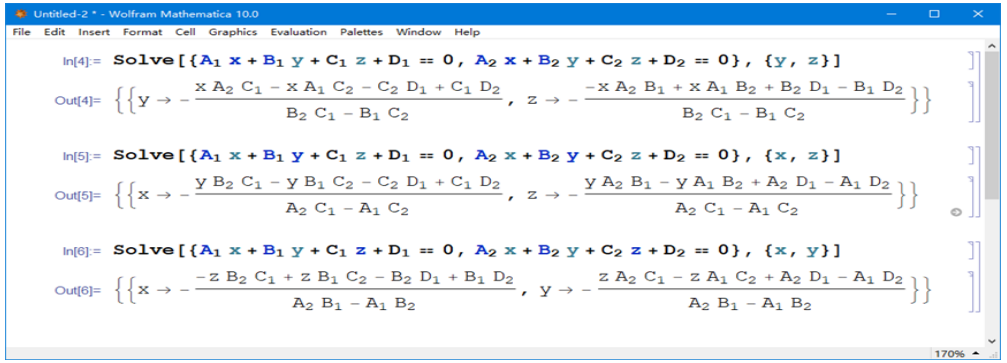
4. 平面

- **toFormula**函数的作用是根据P和N获取直线标准方程中的参数值。
- **zPlane**函数用于生成垂直于坐标系Z轴的平面，这个函数在后面对三维模型切片时会用到。注意zPlane是静态函数，使用类名即可调用。
- **intersect**函数的功能是计算当前平面和另一平面的截交线，如果两个平面不平行（或不重合），返回Line类型的交线，反之返回None，即没有交线。

intersect函数实现原理

$$\begin{cases} A_1x + B_1y + C_1z + D_1 = 0 \\ A_2x + B_2y + C_2z + D_1 = 0 \end{cases} \tag{2-14}$$

- 首选通过两平面法向判断两平面**是否平行**。如果不平行，则求解联立的方程(2-14)
- **不定方程**。具体求解时可将x、y、z中三个未知数中的一个看成已知（如x），另两个看成未知（如y、z），如无解，则将另两未知数看成已知。
- 推荐使用**Mathematica**软件求解。



5. 几何算法

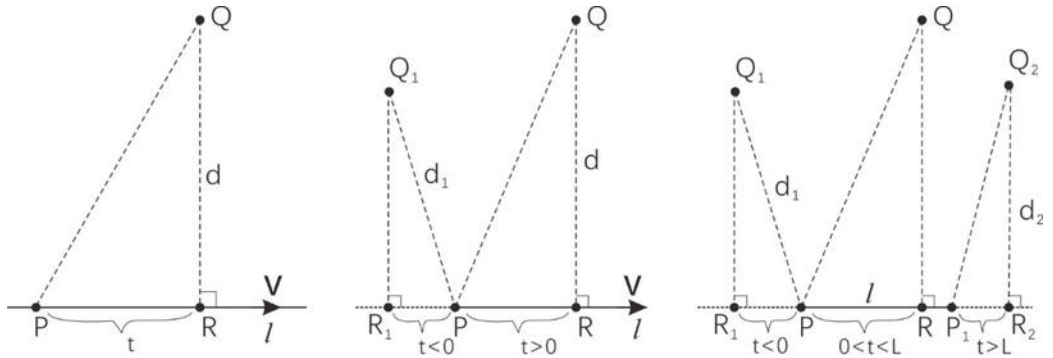
• 距离计算：点线距离

- 对空间中已知的任意一点Q和直线 $l(P, \mathbf{V})$，其中P为 l 经过点，**V**为其单位方向向量。记点Q在直线 l 上的投影为R，设P、R两点距离为t，则向量**PR** = **V**·t。在三角形QPR中，向量**QR** = **QP** + **PR**，即**QR** = **QP** + **V**·t。又因为**QR** ⊥ **V**，所以两向量点乘为0，即：

$$(\mathbf{QP} + \mathbf{V} \cdot t) \cdot \mathbf{V} = 0, \tag{2-18}$$

$$t = -\frac{\mathbf{QP} \cdot \mathbf{V}}{\mathbf{V}^2} = \mathbf{PQ} \cdot \mathbf{V}, \tag{2-19}$$

- t值为**PQ**和**V**两向量点乘结果。投影点R = P + V·t，QR长度即为点Q到直线l距离。



(a) 点到直线距离　(b) 点到射线距离　(c) 点到线段距离

分类讨论

- 如果*l*为射线，还要判断Q的投影R是否在*l*上。根据上面推导，显然，如果t ≥ 0，那么R落在*l*上，否则R落在*l*外。进一步地，如果t ≥ 0，则QR长度即为Q到l的距离，否则QP长度为Q到*l*的距离。
- 如果*l*为线段，记P、P_1为其两端点，长度为L。显然，如果0 ≤ t ≤ L，那么R落在线段上，此时QR长度即为Q到*l*的距离；如果t ≤ 0，QP长度即为Q到l的距离；如果t ≥ L，QP_1长度即为Q到*l*的距离

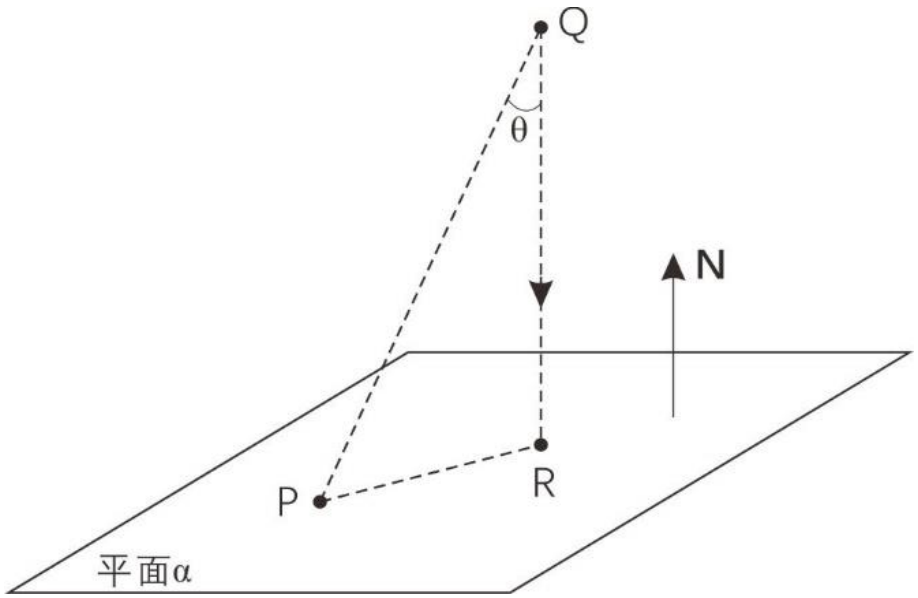
5. 几何算法

• 距离计算：点面距离

- 对空间中已知的任意一点Q和平面α(P，N)，其中P为α经过点，N为其单位法向量。记Q在平面上的投影为R，在三角形PQR中，点P、Q坐标已知，即向量**PQ**已知，向量**QR**方向和N相同（或相反），记∠PQR为θ，则

$$\theta = \arccos\left(\frac{\mathbf{PQ}\cdot\mathbf{N}}{|\mathbf{PQ}|}\right), \qquad (2\text{-}20)$$

- 进而QR = PQ·cos(θ)，QR即为点到平面距离。



5. 几何算法

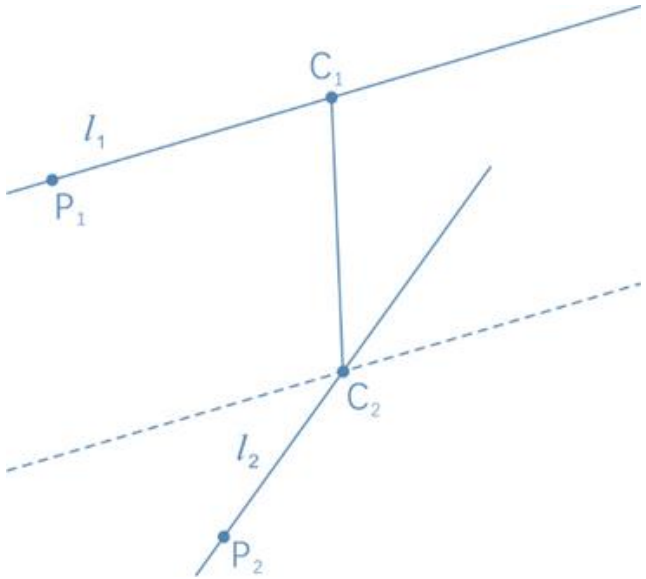
• 距离计算：线线距离

- 这里只讨论两异面直线距离。记两条异面直线为$l_1(P_1, \mathbf{V_1})$和$l_2(P_2, \mathbf{V_2})$。设C_1C_2为其公垂线，则C_1C_2长度即为两异面直线距离。先不急于去求C_1C_2长度，如果将直线l_1从C_1点平移到C_2点，同时将l_2从C_2点平移至C_1点，这样可以得到两张平行平面，且平面法向量为N，则

$$\mathbf{N} = \mathbf{V_1} \times \mathbf{V_2}, \qquad (2\text{-}21)$$

- 此时求取异面直线的距离转化为**求取两平行平面的距离**。进一步地，可以得到，两平行平面的距离等于**点P_1到由P_2和N确定的平面的距离**。这样就将求取异面直线的距离成功转化为已经解决的求取点面距离。

- 在式(2-21)中，如果N为零向量，即l_1平行于l_2，此时问题等价于求取点P_1到直线l_2距离。



异面直线距离计算

5. 几何算法

• 距离计算：线面距离

- **直线和平面距离**，如果直线的方向向量和平面的法向量垂直，那么直线到平面的距离就是直线上一点到平面的距离，否则直线到平面距离为0。
- **射线和平面距离**，和直线到平面距离类似，所不同是，如果射线方向向量和平面不垂直，那么需要考虑射线方向走势，是向着平面还是远离平面。可以采用试探的方法，除射线端点外，再在射线上取一个点，然后分别计算这两个点到平面的距离。如果端点到平面距离较大，那么该距离就是射线到平面距离，否则射线到平面距离为0。
- **线段到平面距离，**相对来说较为复杂，需要判断线段两端点是否位于平面同一侧。首先可以根据式(2-16)建立如下判别式：

$$f(x,y,z) = A(x-x_0) + B(y-y_0) + C(z-z_0). \qquad (2\text{-}22)$$

然后分别将线段两端点代入上式，得到 f_1 和 f_2，如果 $f_1 \quad f_2 \leq 0$，则线段到平面距离为0，否则取两端点到平面距离较小者为线段到平面距离。

5. 几何算法

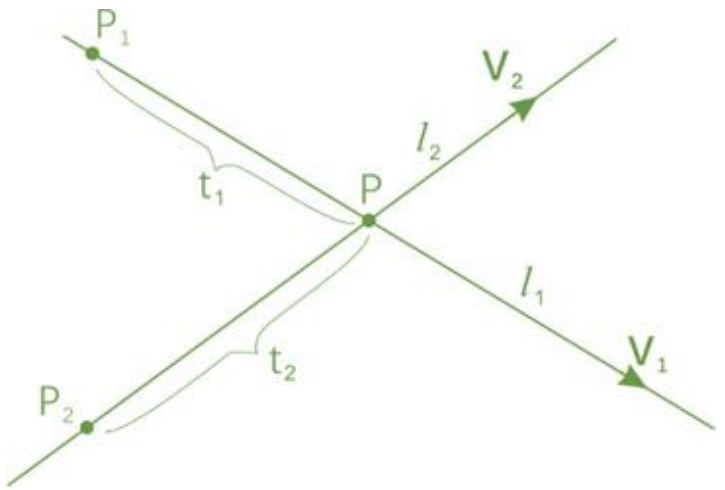
• 求交计算：线线求交

- 考虑一般情况，记两条位于同一平面上的直线分别为 $l_1(P_1, \mathbf{V_1})$ 和 $l_2(P_2, \mathbf{V_2})$。记两直线交点为P，由直线点向式联立可得：

$$\begin{cases} P = P_1 + \mathbf{V_1} \cdot t_1 \\ P = P_2 + \mathbf{V_2} \cdot t_2' \end{cases} \qquad (2\text{-}23)$$

- 其中t_1、t_2分别为P到P_1、P_2的长度参数（可为负），只要求得t_1、t_2值，便可获得交点P坐标。根据式(2-23)进一步可得：

$$\mathbf{P_1P_2} = \mathbf{V_1} \cdot t_1 - \mathbf{V_2} \cdot t_2. \qquad (2\text{-}24)$$



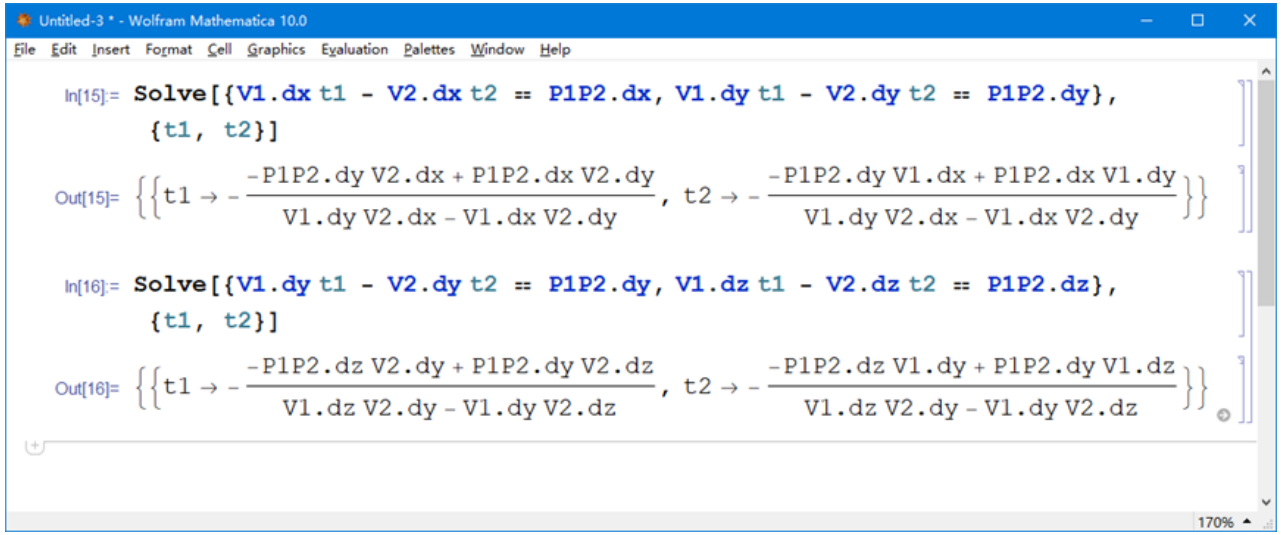
线线求交计算

5. 几何算法

- 求交计算：**线线求交**
  - 考虑到式(2-24)中向量有3个维度，可得：

$$\begin{cases} \mathbf{P_1P_2}.dx = \mathbf{V_1}.dx \cdot t_1 - \mathbf{V_2}.dx \cdot t_2 \\ \mathbf{P_1P_2}.dy = \mathbf{V_1}.dy \cdot t_1 - \mathbf{V_2}.dy \cdot t_2. \\ \mathbf{P_1P_2}.dz = \mathbf{V_1}.dz \cdot t_1 - \mathbf{V_2}.dz \cdot t_2 \end{cases} \qquad (2\text{-}25)$$

  - 对式(2-25)，可以**任取两个等式**借助Mathematica软件求解t_1、t_2。如果分母为0，则**取另外两个等式求解**。如果分母依然为0，说明两条直线平行，无交点。在解得t_1、t_2后，代回式(2-23)，可得两直线交点坐标

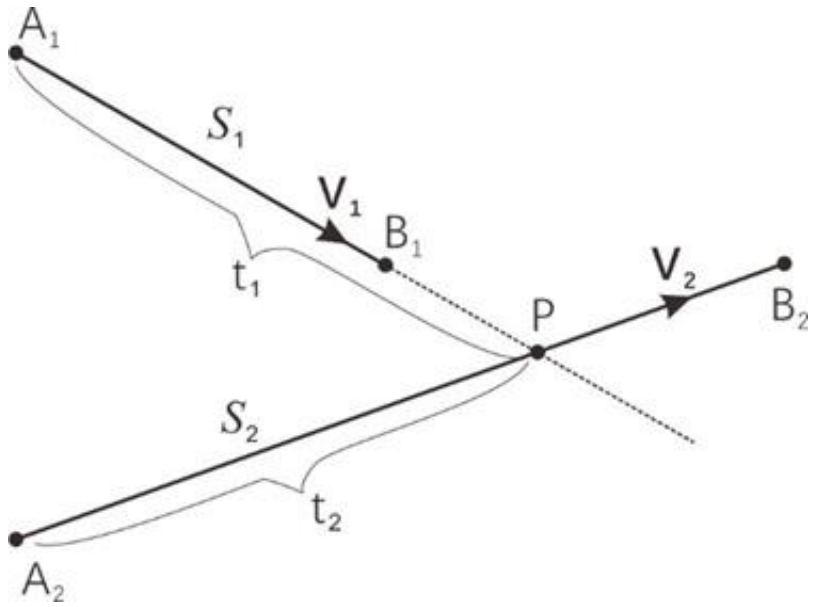


5. 几何算法

- 求交计算：**线线求交**

- 两**线段求交**问题是两直线求交问题的特例。记两线段为$S_1(A_1, B_1)$和$S_2(A_2, B_2)$，另记$\mathbf{V_1} = \mathbf{A_1B_1}/|\mathbf{A_1B_1}|$，$\mathbf{V_2} = \mathbf{A_2B_2}/|\mathbf{A_2B_2}|$，$P_1 = A_1$，$P_2 = A_2$，两线段长度分别为$L_1$和$L_2$，根据这些条件，借鉴以上两直线求交方法可得$t_1$、$t_2$值以及交点P坐标。**如果$0 \le t_1 \le L_1$且$0 \le t_2 \le L_2$，那么直线交点同时落在两线段上，此时两线段相交**；否则两线段不相交。

- 此外，针对直线和线段、直线和射线、射线和线段以及射线和射线的相交问题，同样可以通过判断t_1、t_2取值来判断两线之间是否存在交点。



5.几何算法

- 求交计算：**线面求交**

- 针对线面求交，先来看一下**直线和平面的求交问题**。记直线和平面分别为 $l(P_0, \mathbf{V})$ 和 $\alpha(P_1, \mathbf{N})$，记两者交点为P，那么$P = P_0 + \mathbf{V}\cdot t$。这里依然采用向量法进行分析推导，由于落在平面上，那么P和平面上任意点的连线必然垂直于平面法向量，即$\mathbf{PP_1}\cdot\mathbf{N} = 0$，代入P点坐标推导可得：

$$t = \mathbf{P_0P_1}\cdot\mathbf{N}/\mathbf{V}\cdot\mathbf{N}. \qquad (2\text{-}26)$$

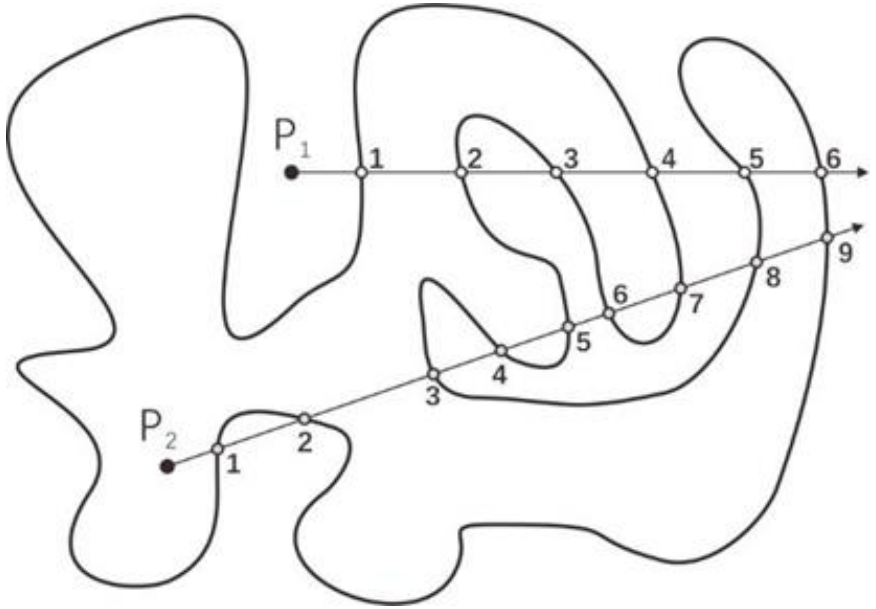
- 如果**V**和**N**垂直，分母为0，此时直线和平面平行，无交点；否则根据计算出的t值可确定交点坐标。

- 针对射线和平面求交以及线段和平面求交问题，可以借鉴处理上述线线求交的方法，先计算射线或线段所在直线和平面的t值以及交点坐标，然后对t的取值进行判断，从而确定交点是否在射线或线段上。

5. 几何算法

• 点和多边形关系判断

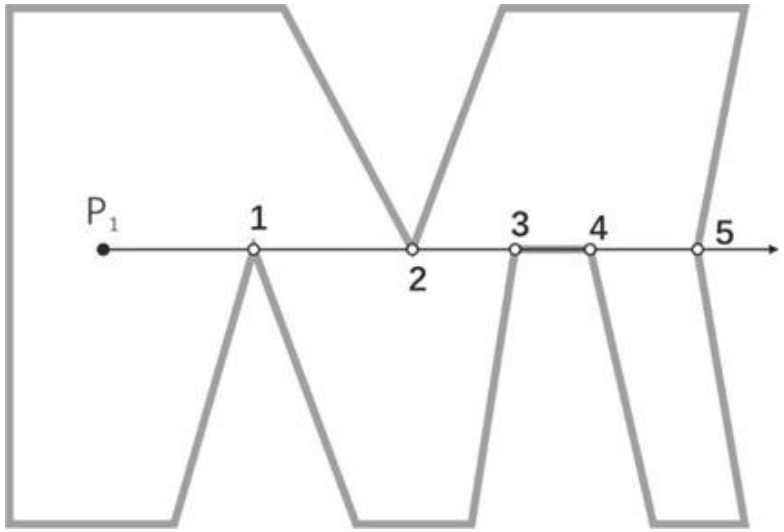
- 平面上（以XY平面为例），点和多边形的关系包括点在多边形内、点在多边形上以及点在多边形外3种情形。
- 如果不考虑点落在多边形上的情形，该问题可用**射线法**解决，即从该点向无穷远处做一条射线，统计射线和多边形的**穿过次数**：**如果穿过次数为奇数，那么该点落在多边形内，如果穿过次数为偶数，那么该点落在多边形外**。



5. 几何算法

• 点和多边形关系判断

- 结合之前介绍的射线和线段的求交方法，可以让射线和多边形中的所有线段求交，获取交点个数，进而统计穿过次数。
- 但仅仅这样是不够的，因为**交点个数并不等价于穿过次数**，如果射线在线段端点处相交，同一位置的交点将被统计了2次，这在某些情况下是对的，如图中的情况1和情况2，也另一些情况下是错的，如图中的情况5。
- 为了避免上述情况的发生，在设计算法时，将射线和多边形的求交问题分成两部分：一是射线和不含**两端点**的线段求交，二是射线和多边形**顶点**求交。

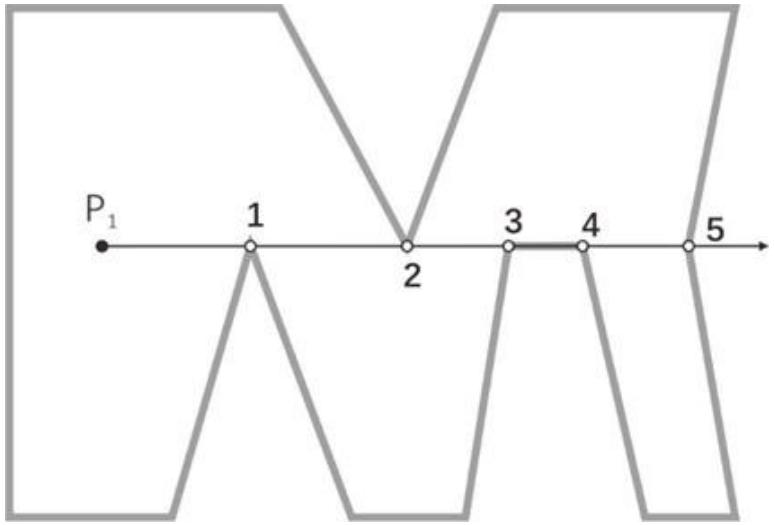


5. 几何算法

• 点和多边形关系判断

- **射线和线段（不含两端点）的求交**。参照两直线求交的方法，首先计算出参数t_1、t_2，其中t_1对应射线，t_2对应线段。如果$t_1 = 0$，表明交点落在射线起点，此时可以立马判定点在多边形上，无需进一步计算。如果$t_1 > 0$且$0 < t_2 < L$，其中L为线段长度，那么射线和线段（不含两端点）相交，此时射线和多边形穿过次数加1。

- **射线恰好在多边形顶点处穿过**。这个问题相对复杂，需要**判断顶点两边线段的走势**。如果两边在射线同一侧，那么可以认为穿过次数为0（或2），如图中的情况1和情况2；如果两边分别在射线两侧，那么认为穿过次数为1，如图中的情况5；但是如果两边中有一边和射线重合，如图中的情况3和情况4，则需进一步判断下一条边的位置。



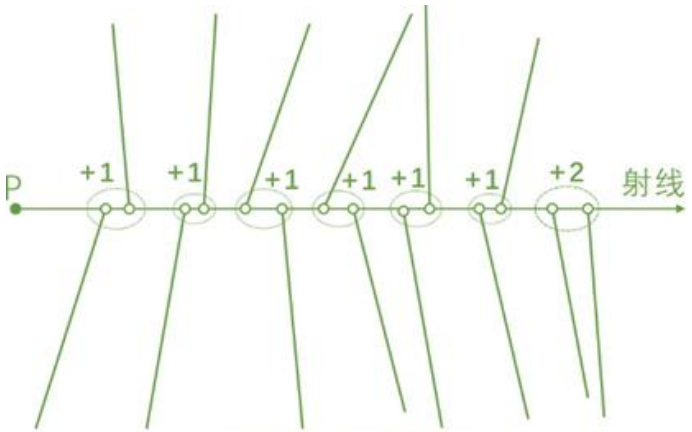
5. 几何算法

• 点和多边形关系判断

- 针对**射线恰好在多边形顶点处穿过**的情形：退一步思考，首先统计所有顶点位于射线上的线段，如果某条线段和射线重合，则认为该线段两端点都不在射线上。然后对这些线段进行**配对**。如图所示:

（1）不必理会线段之间的相对位置，直接对一条位于射线一侧的线段和一条位于射线另一侧的线段进行配对，配对成功后穿过次数加1。

（2）对那些位于射线同一侧的线段对，要么认为穿过次数为0，要么认为穿过次数为2，对最终的穿过次数奇偶性不影响。



- 射线的构造。在以上讨论中，认为射线的方向是任意的，但这样会给实际计算带来不少麻烦，比如判断点和射线的位置关系。简单的做法就是在不影响结果的前提下**找一个特例**，而最简单的特例就是**让射线方向和x轴方向平行**，这样要判断点和射线的相对关系只需比较两者y坐标高低即可。

5. 几何算法

- 几何算法模块**GeomAlgo**
  - 根据以上介绍的实体距离、实体求交、点和多边形关系判断等算法内容，在GeomAlgo.py文件中实现相应函数。

GeomAlgo.py　几何算法模块，封装一些基础计算几何算法　2-11

```
1   ……                                          # 导入必要模块
2   def nearZero(x):                            # 判断一个数是否接近0
3       return True if math.fabs(x) < epsilon else False    # 判断该数和给定极小数关系
4   def distance(obj1, obj2):                   # 实体距离计算
5       if isinstance(obj1, Point3D) and isinstance(obj2, Line):        # 情况1：点和直线距离计算
6           P, Q, V = obj2.P, obj1, obj2.V
7           t = P.pointTo(Q).dotProduct(V)
8           R = P + V.amplified(t)
9           return Q.distance(R)
10      elif isinstance(obj1, Point3D) and isinstance(obj2, Ray):       # 情况2：点和射线距离计算
11          P, Q, V = obj2.P, obj1, obj2.V
12          t = P.pointTo(Q).dotProduct(V)
13          if t >= 0:
14              R = P + V.amplified(t)
15              return Q.distance(R)
16          return Q.distance(P)
17      elif isinstance(obj1, Point3D) and isinstance(obj2, Segment):   # 情况3：点和线段距离计算
18          Q, P, P1, V = obj1, obj2.A, obj2.B, obj2.direction().normalized()
19          L = obj2.length()
20          t = P.pointTo(Q).dotProduct(V)
```

……

本章小结

- 这一章使用Python语言，从零开始，搭建了一个基础计算几何库：
  - 建立了点、向量、矩阵、直线、射线、线段、多段线、平面等几何实体类；
  - 实现了实体距离计算、求交计算以及点和多边形关系判断等若干几何算法。
- 这些基础类和算法是后续章节进一步展开的基础。在今后的学习中，希望同学们可以不断积累，持续地扩充自己的几何库。





思考题

1. 在计算几何中，为什么点和向量一般使用齐次坐标表示?
2. 简述四阶方阵中，各子区域中各个元素的作用。
3. 在三维空间中，如何判断两个不同的点相等或重合?
4. 在三维空间中，如何判断两个向量是否指向同一方向?
5. 实现GeomBase模块中Matrix3D类的逆矩阵求取函数。
6. 简述在直线、射线、线段类的初始化函数中对参数进行拷贝的原因。
7. 列举至少两种方法，判断一个平面多边形的顺逆时针方向。
8. 设计一种算法，在平面上，快速报告点云中落在多边形中点的方法。
9. 设计一种算法，快速计算平面上两个多边形之间的最近距离。